

Report

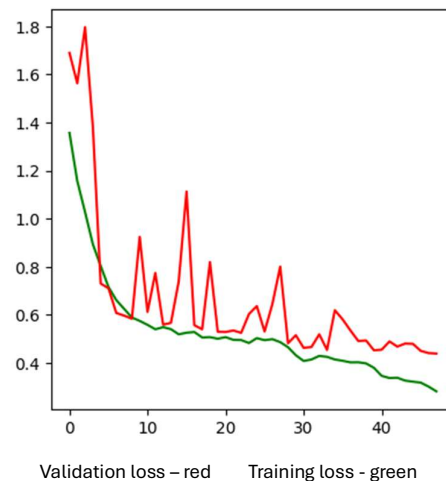
Kaggle Submissions:

EXPLORATORY DATA ANALYSIS:

I have noticed that the values in the data are between 0 – 255 and by manual verification I figured out the dataset is of images and it has a data imbalance of classes 0 and 1 of ratio 9:1 so I have used data augmentation as my strategy to avoid biased prediction towards the class 1.

OVERFITTING ISSUE:

Since I have a large no of images that are augmented the model overfitted to the augmented data. I concluded this by check the loss and accuracy on original distribution as well as augmented data if augmented data loss is less than or accuracy is greater than that of original distribution I have concluded it as overfitting. Initially model's complexity was high so model has overfitted so I have reduced complexity I have concluded this by checking whether validation loss is peaking or not while the training loss is going down like the following



Methods used:

I have used K-Fold Cross Validation and I have used l1 and l2 norm to avoid exploding gradients and I have taken care of model's complexity. I have used global average pooling in the first model thinking it might help solve model overfitting issue. And I have used following methods.

- 1)Early Stopping
- 2)Callback
- 3) Reduce LR on Plateau
- 4)Dropout Layers

```
class myCallBack(tf.keras.callbacks.Callback):
    def on_epoch_end(self, epoch, logs = {}):
        if(logs.get('accuracy') > 0.95):
            print("Model has acquired an accuracy of 85% on training dataset")
            self.model.stop_training = True
dynamiclr = tf.keras.callbacks.ReduceLRonPlateau(
    monitor='val_loss',
    factor = 0.5,
    patience = 3,
    verbose = 1,
    mode = 'auto',
    min_delta = 0.001,
    cooldown = 0,
    min_lr = 1e-6
)
stopearly = tf.keras.callbacks.EarlyStopping(
    monitor = 'val_loss',
    start_from_epoch=6,
    patience = 2,
    verbose = 1,
    min_delta = 0.0001,
    restore_best_weights=True
)
```

```

model = Sequential([
    Conv2D(32,(3,3)),
    BatchNormalization(),
    MaxPooling2D((2,2)),
    Dropout(0.25),

    Conv2D(64,(3,3),activation = 'relu',kernel_regularizer=l1(0.001)),
    BatchNormalization(),
    MaxPooling2D((2,2)),
    Dropout(0.25),

    Conv2D(128,(3,3),activation='relu',kernel_regularizer=l2(0.001)),
    BatchNormalization(),
    MaxPooling2D((2,2)),
    Dropout(0.25),

    tf.keras.layers.GlobalAveragePooling2D(),

    BatchNormalization(),
    Dropout(0.25),
    BatchNormalization(),
    Dense(units = 64,activation = relu),
    BatchNormalization(),
    Dense(units = 2,activation = softmax)
])

```

2)CNN model

```

model = Sequential([
    Conv2D(32,(3,3) , activation = relu,input_shape = (32,32,3)),
    BatchNormalization(),
    Dropout(0.2),
    Conv2D(32,(2,2),activation = relu,kernel_regularizer=l1(0.001)),
    BatchNormalization(),
    Flatten(),
    Dense(256 , activation = relu),
    Dense(256 , activation = relu),
    Dense(10 , activation = softmax)
])

```

Justification:

I have used a CNN model to since the dataset is of images and I found the Kaggle Public score to be decently good about 0.85 to 0.90 for both the model however I had other models that scored better taking into consideration of overfitting I thought these models would have performed but has not been true in private score due to severe overfitting in the first model and use of excessive augmentation of data

Model I have tried:

- 1)CNN
- 2)Visual Transformer (Failed due to less amount of random predicting the data and excessive augmentation)
- 3)K-Nearest Classifier (Misjudged performance due to public score)

Mistakes I have done:

Use of public score as an analytic metric not considering F1-Score

Linear Regression:

Data Analysis:

I normalized data by using normal distribution and saved the mean and variance in form of metrics data frame and did a test split of 0.1 for r2_score validation

N features: 15

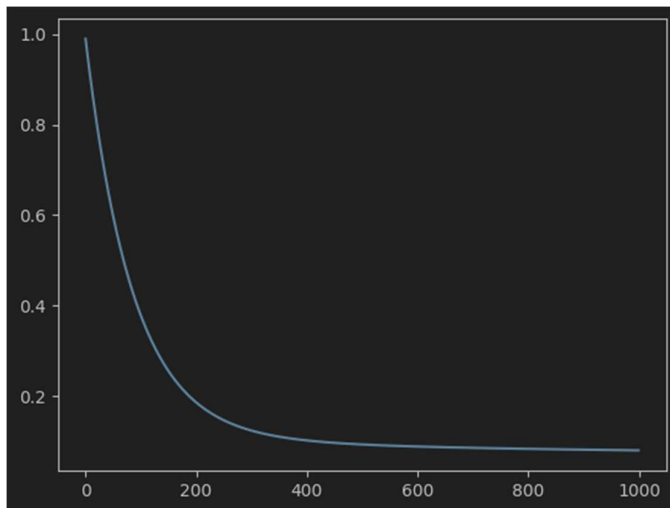
N datapoints: 60000

Training Details:

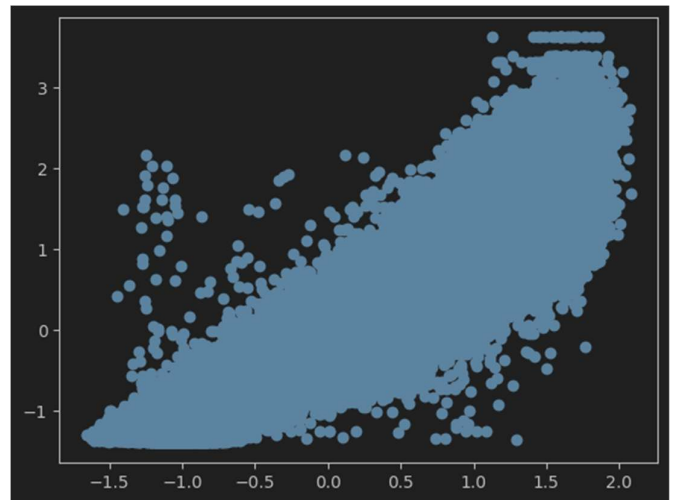
R2 Score achieved 0.92.

Model Status: Converged.

Final MSE = 0.08



Training Curve



Scatter plot between predictions and original data

Justification:

Model has achieved a fairly good R2 Score and the error in prediction of normalized values is close to 5.6% so model has converged to and there is positive correlation between the predictions and targets. But when it is converted back to the Original distribution I have observed error percentage significantly increases.

Logistic Regression:

Data Analysis:

No of Features: 24

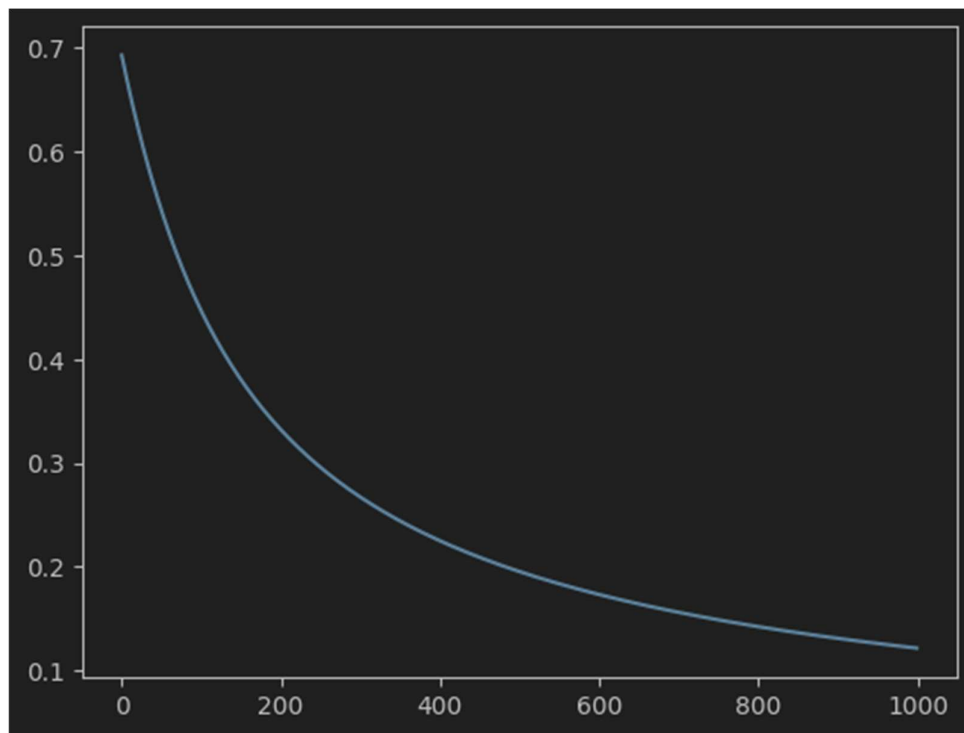
No of datapoints: 50000

Normalized data by using normal distribution and saved the mean and variance in form of metrics data frame and did a test split of 0.1

Training details:

No of iterations: 1000

Final Log loss achieved: 0.1216
F1 Score achieved: 0.99298

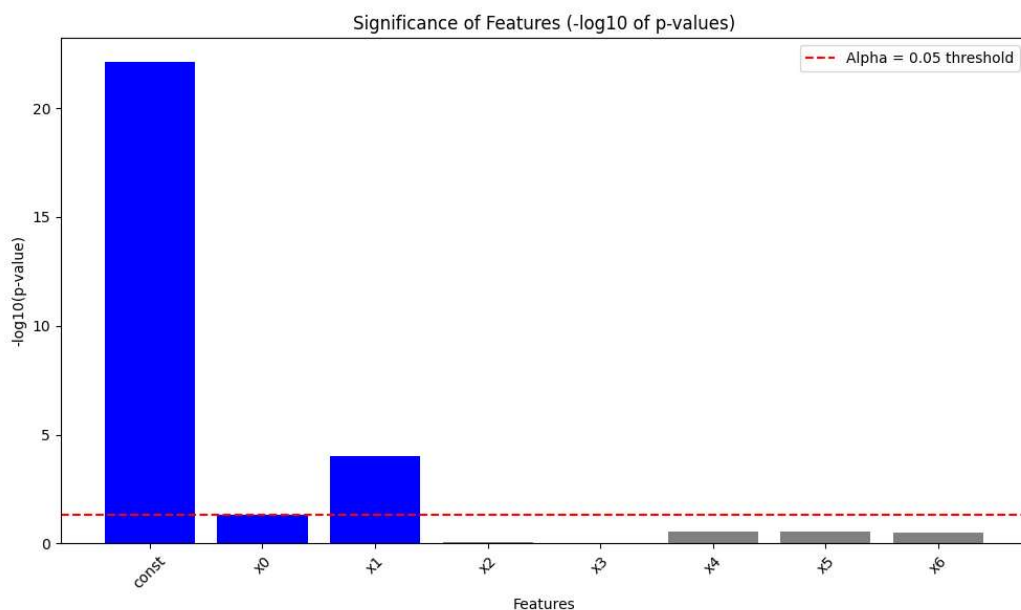


Training loss graph

Polynomial Regression:

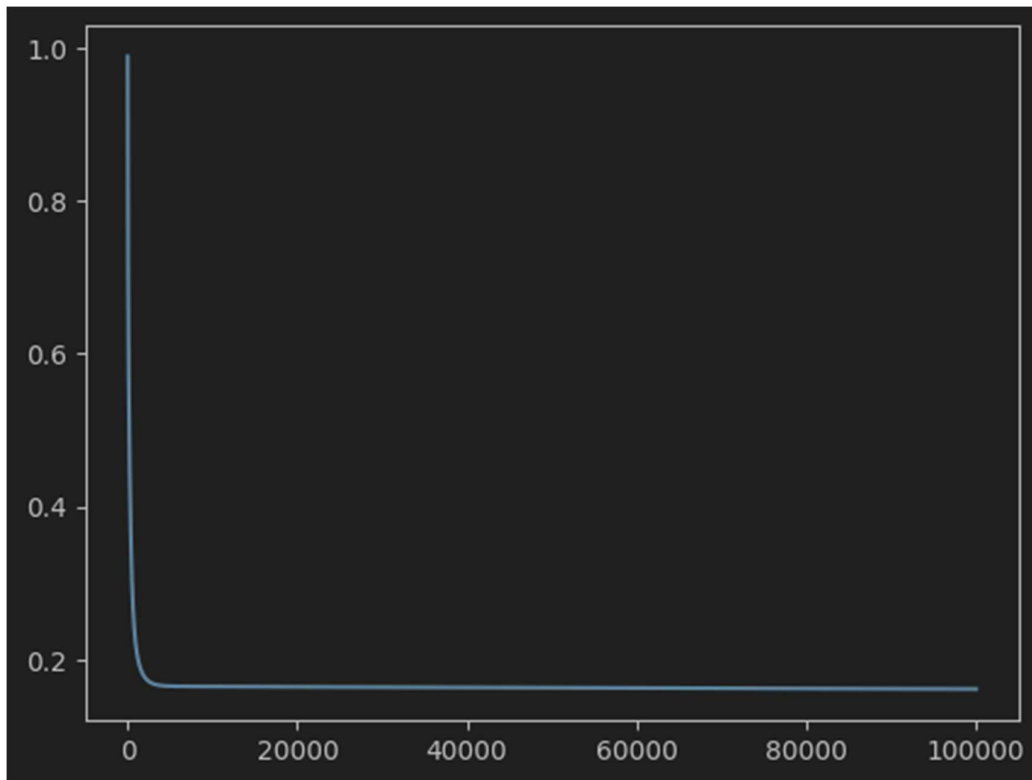
Data analysis:

- 1) Data had some missing values so I cleaned the data
- 2) I have figured using calculation of p-values only feature x_0 and x_1 are significant in determining the target so I have trained model only on the x_0 and x_1 to avoid the floating-point math not to break while transforming the matrix.
- 3)
- 4) Both features as well as target values and overloaded so I normalized both

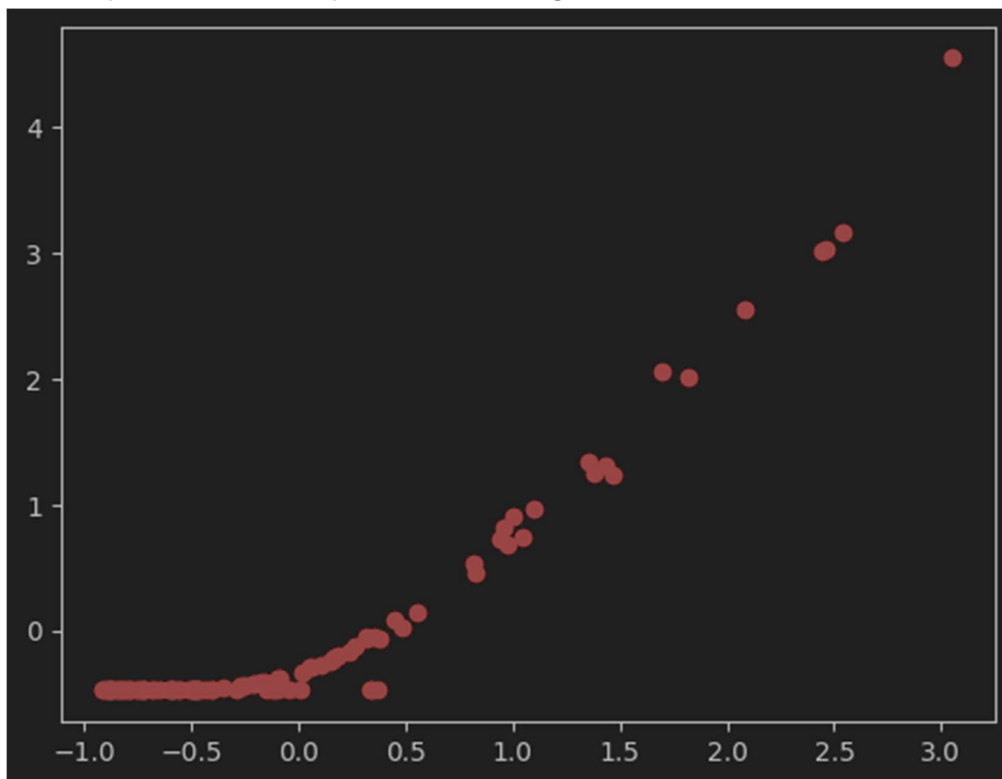


Training Details:

1) Training loss Graph:



1) Scatter plot between the predicted and original data in test data



Regression Neural Network:

Data Analysis:

The data consists of 16000 data samples and I have normalized the data just avoid the discrepancy between features 17 and 20 which are approximately in range 0-45 and 0 – 150 Where all other features including output are approximately in range of 0 -15. I have used a test split of 0.4 for this dataset to check for the F1-Score

Model Structure:

```
model = MLP(  
    (Layer(nin = 20,nout=1,nonlin = False)),  
    )  
)
```

Justification for small complexity of model:

Since my model is based on Neuron object it is structurally creating millions of Neurons which has to cleared by python garbage collector but somehow the python garbage collector just ignores them and they get

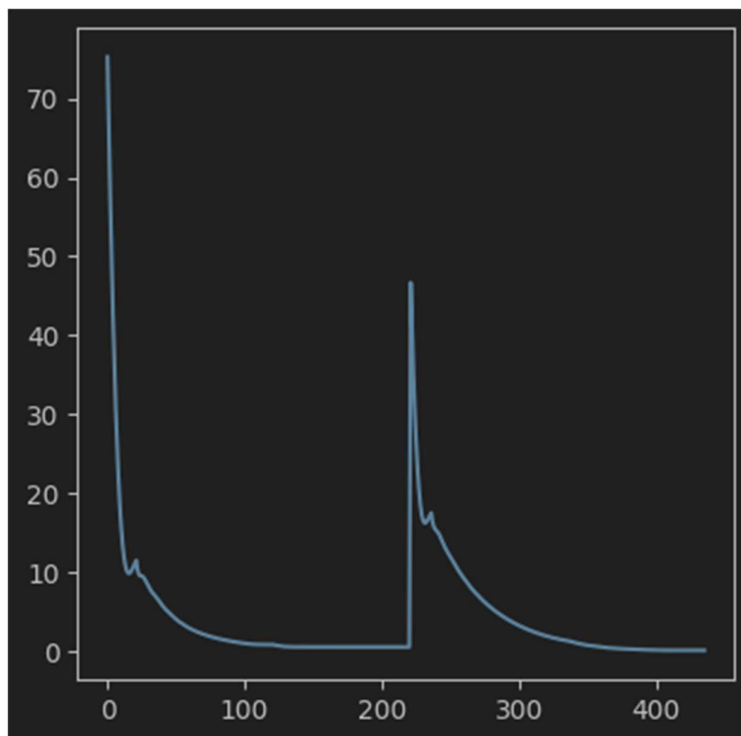
Data analysis:

No of features: 20

No of epochs: 450

Training analysis:

Training Graph:



Justification (for spike in training graph):

There a spike in train process because of miniature Early Stopping I have used in my code

Metrics Achieved:

R2-Score: 0.92

MSE: 0.1750

Justification:

Since my model has an MSE of 0.1750 I think the predictions will be accurate from 0.4183 which on a scale of 0-15 is 2.7% so I think my model has converged

Methods I have tried to use during this training:

1) Miniature Early stopping and Reduce LR on plateau

```
if ((len(loss) > 3 and np.abs((loss[-2] - loss[-1])/loss[-1]) < 0.05) and cost.data < 0.2) and lr*0.8 > 1e-5 ):
    lr = lr*0.8
    print(f"\n reduced lr to {lr}")
if(len(loss) > 3 and loss[-2] < loss[-1]):
    patience += 1
    if (patience > 3 ) or cost.data < 0.01:
        patience = 0
    print("\n Early stopping")
```

Improvisations that can be done:

- 1) Using a slotted data structure decreases the amount of RAM occupied by 40-50%.
- 2) Introducing patience to Reduce LR on plateau.
- 3) Using Batching to train.

Classification Neural Network:

Data Analysis:

The data consists of 16000 data samples and I have normalized the data just to avoid the discrepancy between features 17 and 20 which are approximately in range 0-45 and 0 – 150 whereas all other features are approximately in range of 0 -15 and there are 26 classes. I have used a test split of 0.4 for this dataset

Model Structure:

```
model = MLP(
    (Layer(nin = 20, nout=26, nonlin = True),
      Layer(nin = 26, nout = 26, nonlin = True),
      BatchNorm1d(26),
      Layer(nin = 26, nout = 26)
    )
)
```

Justification:

Initially I was suffering from vanishing gradients problem to avoid that I have used Batch Normalization and ReLU activation before the Batch Normalization layer and since the logits can be anything the high value like 100 are simply killing the back propagation so I used this

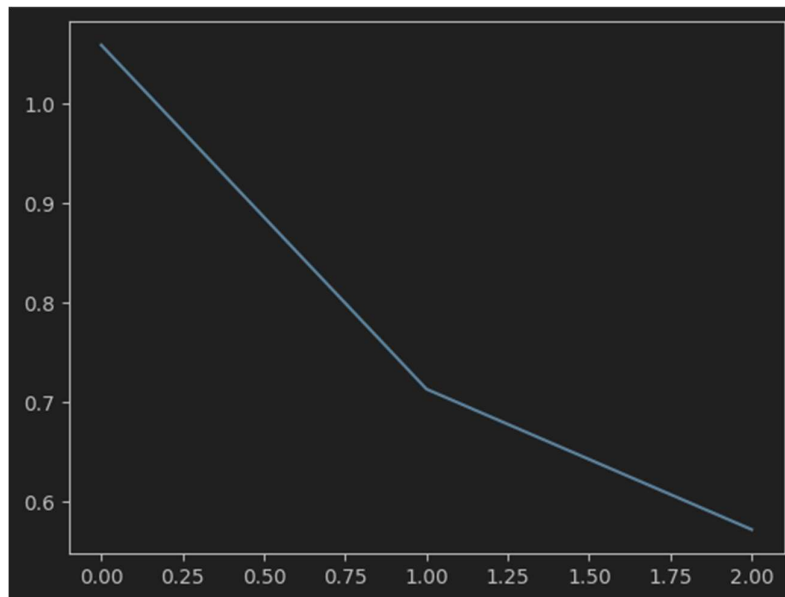
`shifted_logits = [l - max_val for l in logits]` so these are the improvements I have made to make sure that the model doesn't suffer by vanishing gradients problem

Training Analysis:

```
loss += model.fit(xs.to_numpy(), reg_targets.to_numpy(), epochs = 1, batchsize=200, optimizer='adam', lr = 0.01)
```

After experimenting I have found these metrics to be good to train with and then model has converged to a loss value of 0.6

F1-Score: 0.7961



Justification:

I think I have trained it for 2 epochs and over 96 batches and model has achieved a good F1-Score of approximately 0.8 so I think the model has converged

Improvements I have made over Neural network for regression and I am able train a bigger model is because I have made the Neuron Datatype Slotted and then manually destroyed the unnecessary Neurons in RAM after every single batch

```
for p in self.parameters():
    p.children = ()
    p._backward = lambda: None
```

Fixed a bug in `_truediv_` and `_rtruediv_` methods in Neuron Datatype
After:

```
def __truediv__(self, other):
    return self * other**-1

def __rtruediv__(self, other):
    return other * self**-1
```

Before:

```
def __truediv__(self, other):
    other = other if isinstance(other, Neuron) else Neuron(other)
    def _backward():
        self.grad += out.grad/other.data
        other.grad += -(self.data / (other.data ** 2)) * out.grad
    try:
        out = self.data/other.data
        return Neuron(out, children = (self, other), _backward = _backward)
    except ZeroDivisionError:
        out = Neuron(0)
    def _backward():
        self.grad += out.grad/other.data
        other.grad += -(self.data / (other.data ** 2)) * out.grad
    out.children = (self, other)
    out._backward = _backward
    return out
```

Bug Description if other was 0 the graph was just being disconnected so to avoid that I have used the above where every graph sure connected

Initially I used Depth-First Search and then reversed to get the topological order but it broke recursion depth of python so I used Breadth-First Search Kahn's Algorithm to get topological order

Before:

```
def backward(self):
    self.grad = 1
    visited = set()
    topo = list()
    def topo_sort(n : Neuron, visited : set, topo : list):
        if n not in visited:
            visited.add(n)
            for child in n.children:
                topo_sort(child, visited, topo)
            topo.append(n)
    topo_sort(self, visited, topo)
    for n in reversed(topo):
        n._backward()
```

After:

```
topo = []
visited = set()
stack = [(self, 0)] # (node, state) where state=0 -> first time, state=1 -> after children

while stack:
    n, state = stack.pop()
    if state == 0:
        if n in visited:
            continue
        visited.add(n)
        # post-order: push (n,1) then its children with state 0
        stack.append((n, 1))
        for child in n.children:
            if child not in visited:
                stack.append((child, 0))
    else:
        # all children processed; now append to topo
        topo.append(n)

# same as before
for n in reversed(topo):
    n._backward()
```

Multi-Classification:

Data Analysis:

The data consists of 50000 data samples and I have normalized the data and there are 5 classes and there are very few missing values so cleaned them too. I have used a test split of 0.4 for this dataset

I have done the same with multi classification model also but I have changed a few

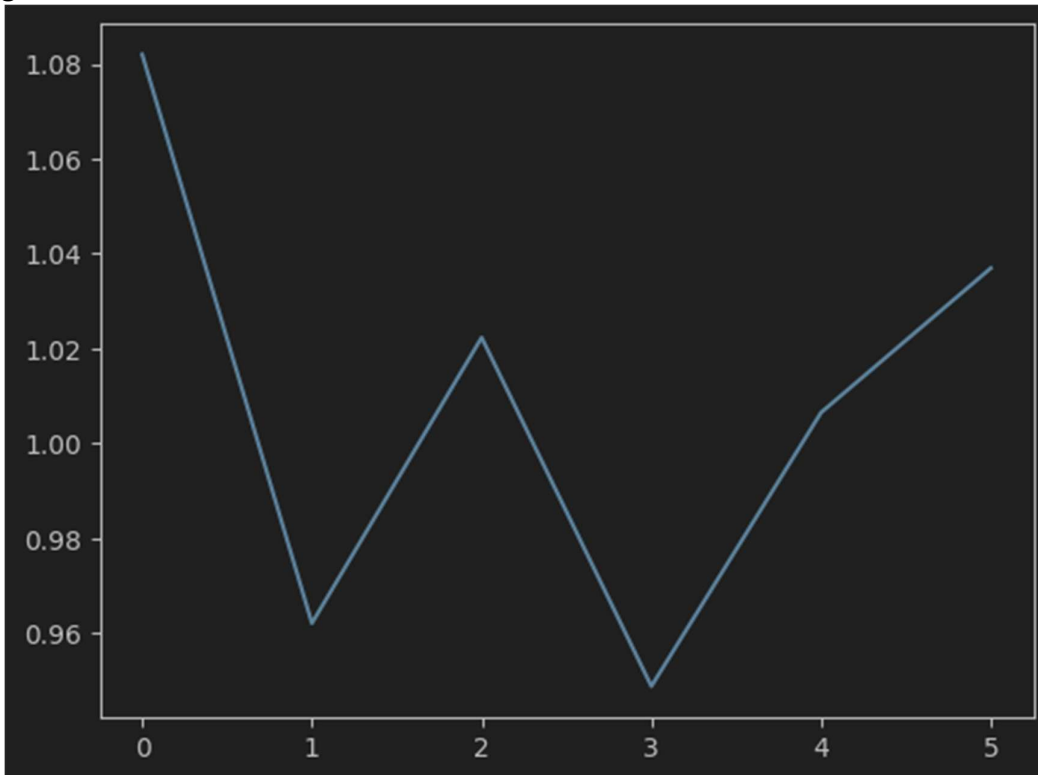
1) Training Setup:

```
loss += model.fit(xs.to_numpy(), reg_targets.to_numpy(), epochs = 1, batchsize= 256 ,optimizer='adam',lr= 0.005)
```

Model Structure:

```
model = MLP(  
    (Layer(nin = 20,nout=10,nonlin = True),  
     Layer(nin = 10,nout = 5, nonlin = True),  
     BatchNorm1d(5),  
     Layer(nin = 5,nout = 5)  
    )  
)
```

Training Loss:



Justification:

I see that the graph over here is fluctuating but I have tried decreasing the learning rate but the loss values keep fluctuating around I have tried see the gradients of params but they to seem fine so I couldn't figure out what was causing an issue. I have trained this model for 5 epochs and 585 batches I feel that model has converged

Metrics Achieved:

F1-Score : 0.5429

K-Means Clustering

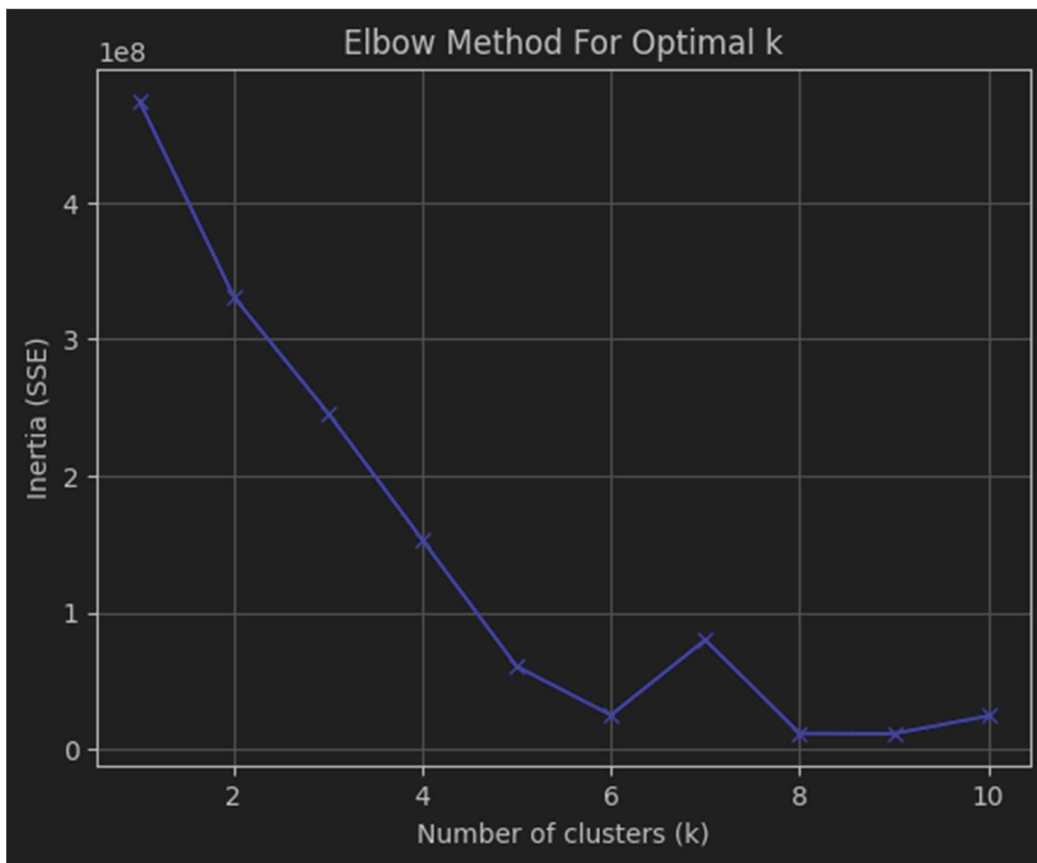
Data Analysis:

There are 6 Features per data point in this dataset and I have not normalized the dataset in this model.

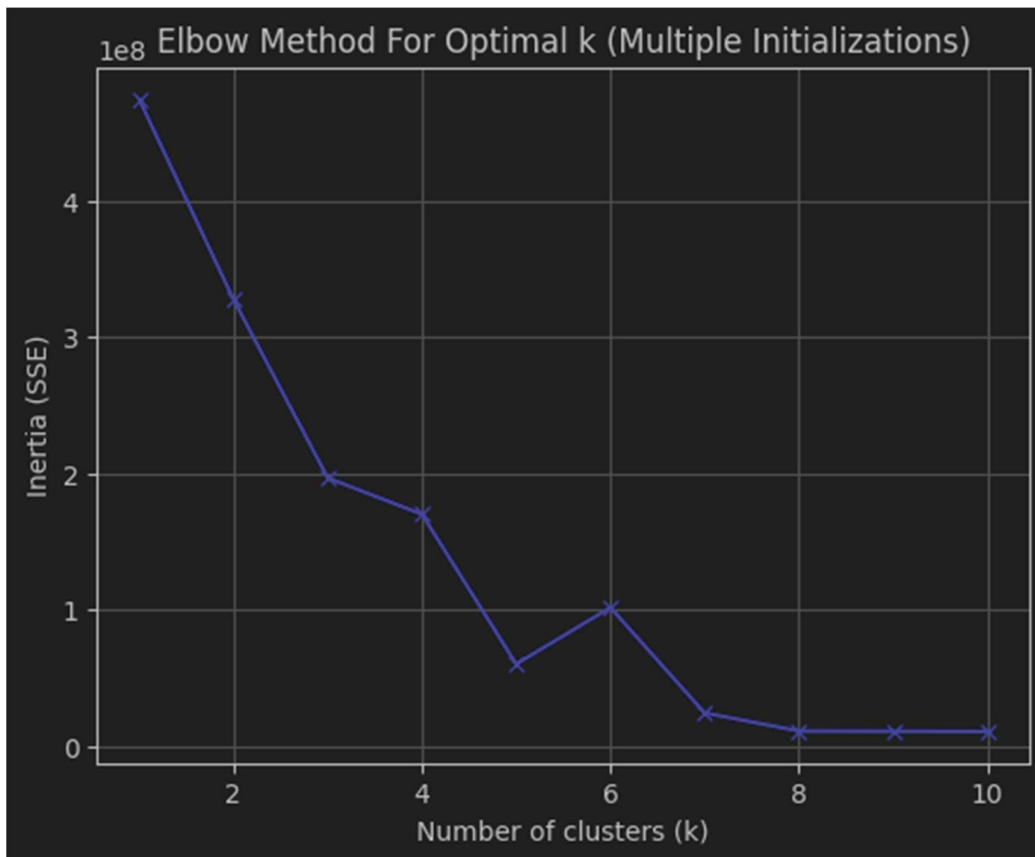
Training Details:

- 1) No.of Epochs = 100

Optimal K (Elbow Method):



Elbow graph for model being initialized one time



Elbow graph after initializing the model for 5 times and inertia = best inertia

So therefore, I conclude the optimal k to be 8.

K-Nearest Neighbours:

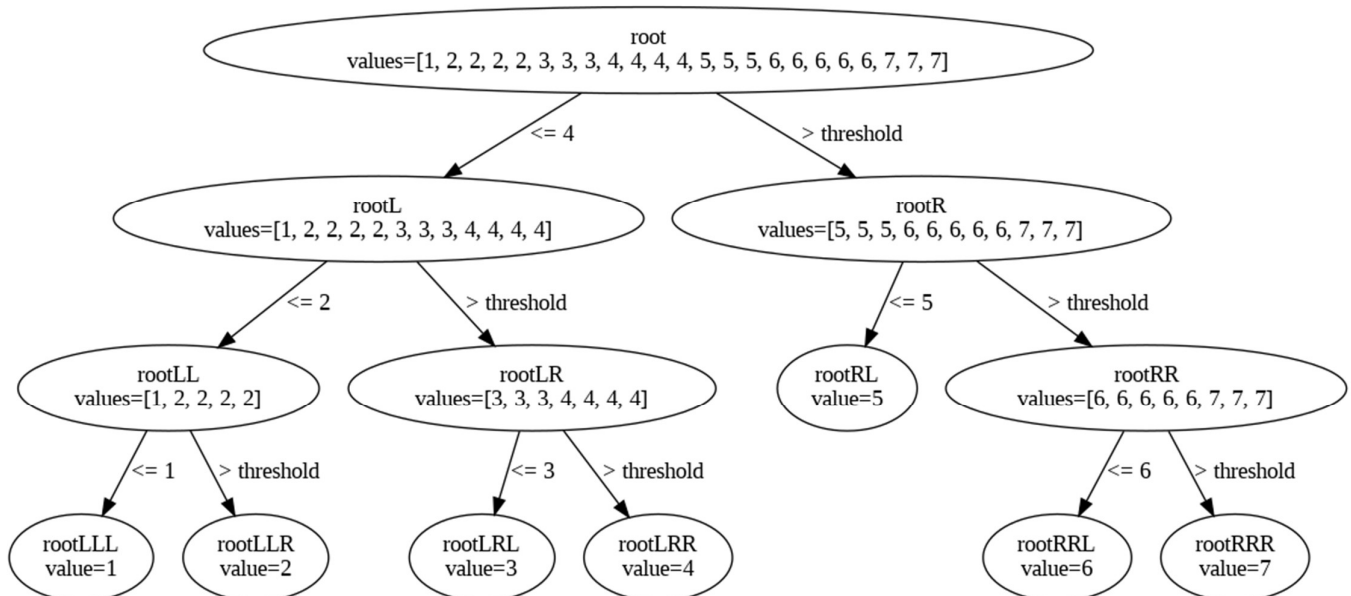
I developed the basic structure and trained the model on a minute datapoints

```
training_data = [[1, 2], [2, 3], [3, 4], [6, 7], [7, 8]]
training_labels = ['A', 'A', 'A', 'B', 'B']
test_point = [4, 5]
k = 3
model = KNearestNeighbours(3)
model.predict(training_data, training_labels, test_point)
[13]
'A'
```

Decision Trees:

I have developed the basic structure and trained on minute datapoints

```
a = [1, 2, 2, 2, 2, 3, 3, 3, 4, 4, 4, 4, 5, 5, 5, 6, 6, 6, 6, 6, 7, 7, 7]
model = DecisionTree()
e = model.build_tree(a)
```



I Hope I have done a good job. I thank all the seniors who have been helping me all these you have made me better all the time. I am made some mistakes during the Kaggle Competition but as we live and learn I would take it as a great lesson and thank you all for allowing me compete in WOC 8.0.